Who Let the Builds Out?

Datadog’s Journey to Remote Execution with Buildbarn

October
2026



DATADOG

Buildbarn Meetup

Who Let the Builds Out?



James Sharpe
Build



Guilhem Charles
CI

Buildbarn Meetup

Who Let the Builds Out?



Andrew Lockwood
Build



Long Võ
CI



Dmitrii Prediger
CI

Agenda

01 Datadog CI

02 How Buildbarn is set up at Datadog

03 Scheduler Tuning

04 Storage contention: failure mode and mitigations

05 Bazel client-side tuning

06 Next Steps

07 Questions

Datadog CI

The Great CI Bottleneck of 2026 is upon us

Datadog CI



Serving thousands of engineers

- GitLab
- GitHub Actions (managed and self-hosted)
- Windows/macOS for agent, SDKs, etc.



Container-based fleet

At peak we run tens of thousands of pods concurrently



Explosive Growth

CI jobs grew by about 80% from November 2025 to now. From running ~20 millions to ~50 millions CI jobs per month.



Bazel Executed Actions Over Time

Billions of Bazel actions executed every week

Datadog CI Ecosystem

Short story of Bazel builds

Datadog CI runs on Kubernetes

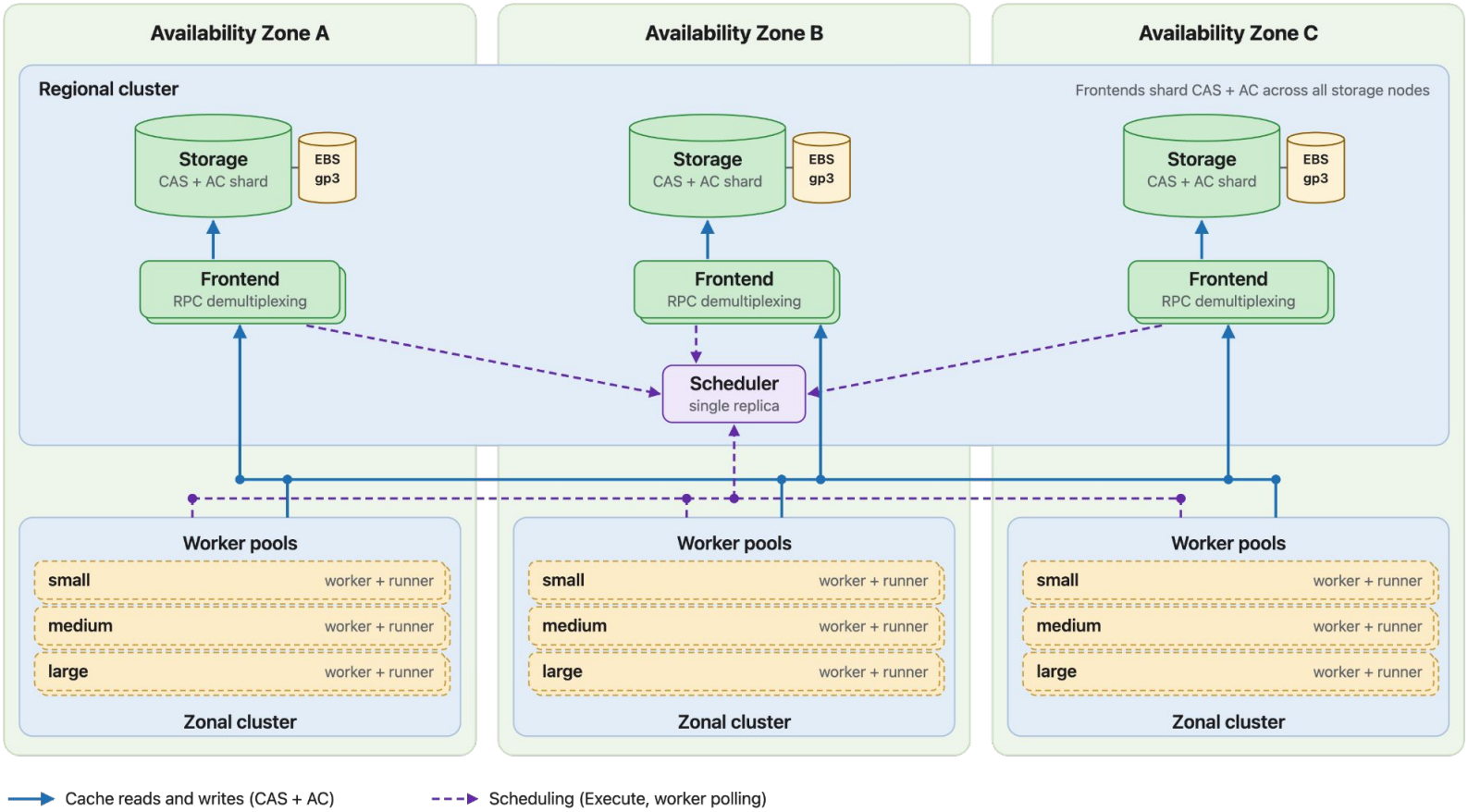
Default model for Datadog CI is to use ephemeral runners: 1 CI job == 1 pod

For bazel that meant a cold start for every job

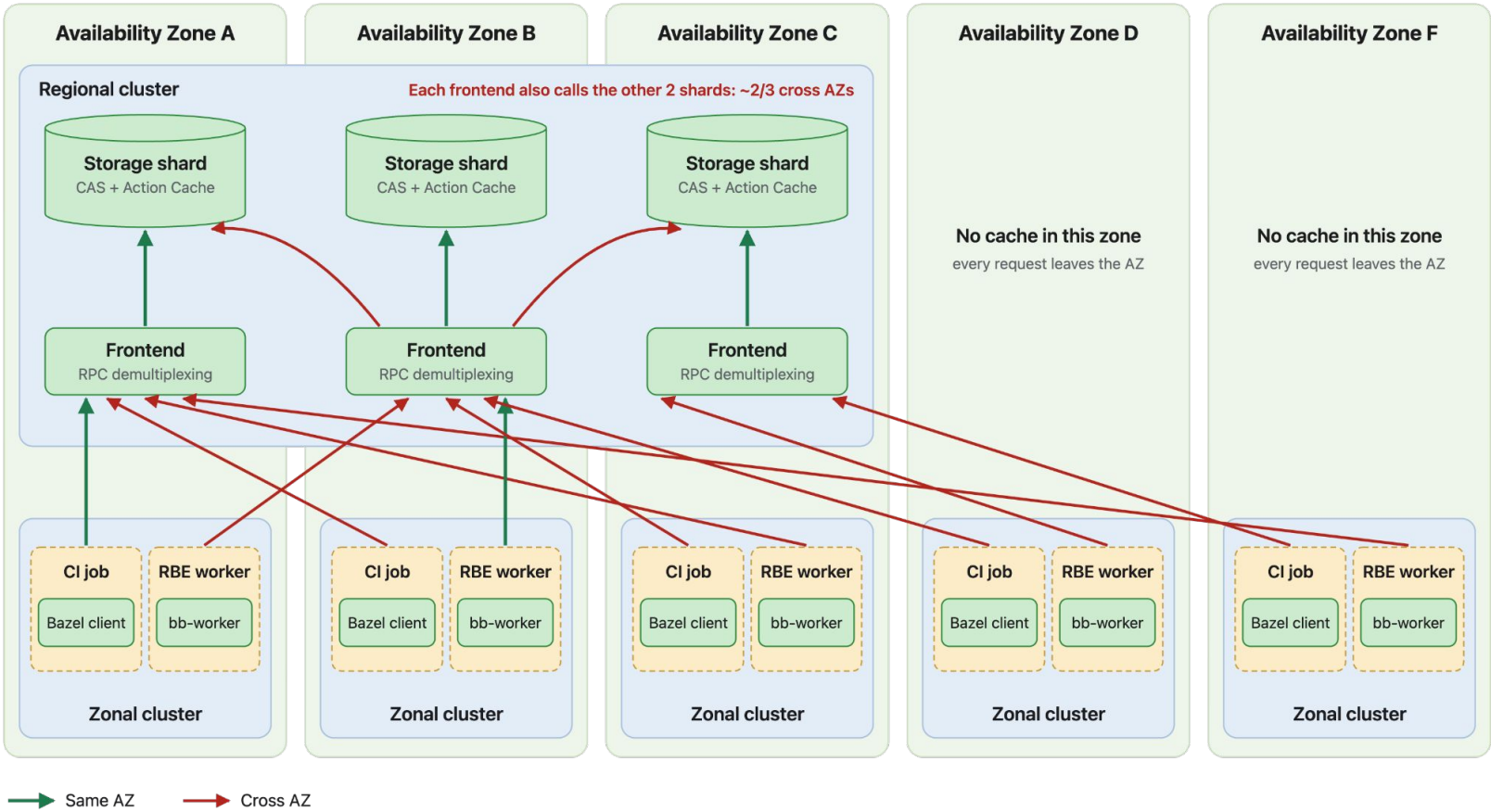
Moved to persistent runners: better cache re-use and warm server

Next logical step was remote execution

How Buildbarn is set up at Datadog


Availability Zone A
Availability Zone B
Availability Zone C
Regional cluster
Frontends shard CAS + AC across all storage nodes
Storage
CAS + AC shard
EBS
gp3
Frontend
RPC demultiplexing
Scheduler
single replica
Worker pools
small
worker + runner
medium
worker + runner
large
worker + runner
Zonal cluster
Cache reads and writes (CAS + AC)
Scheduling (Execute, worker polling)

Problem: The cache covers 3 AZs, CI runs in 5



Workload Collocation

Collocate with the cache, don't spread across zones

Large storage instances are not uniformly available across AZs, so capacity constrains placement. At our scale cross-AZ cache traffic is costly: historically $20–30k/month. Since this diagram was drawn, placement and routing have evolved around those constraints: RBE workers are restricted to cache AZs and use AZ-local frontends.

Not sure if this one or the one before



$20-25k
per month in cross-AZ
data transfer

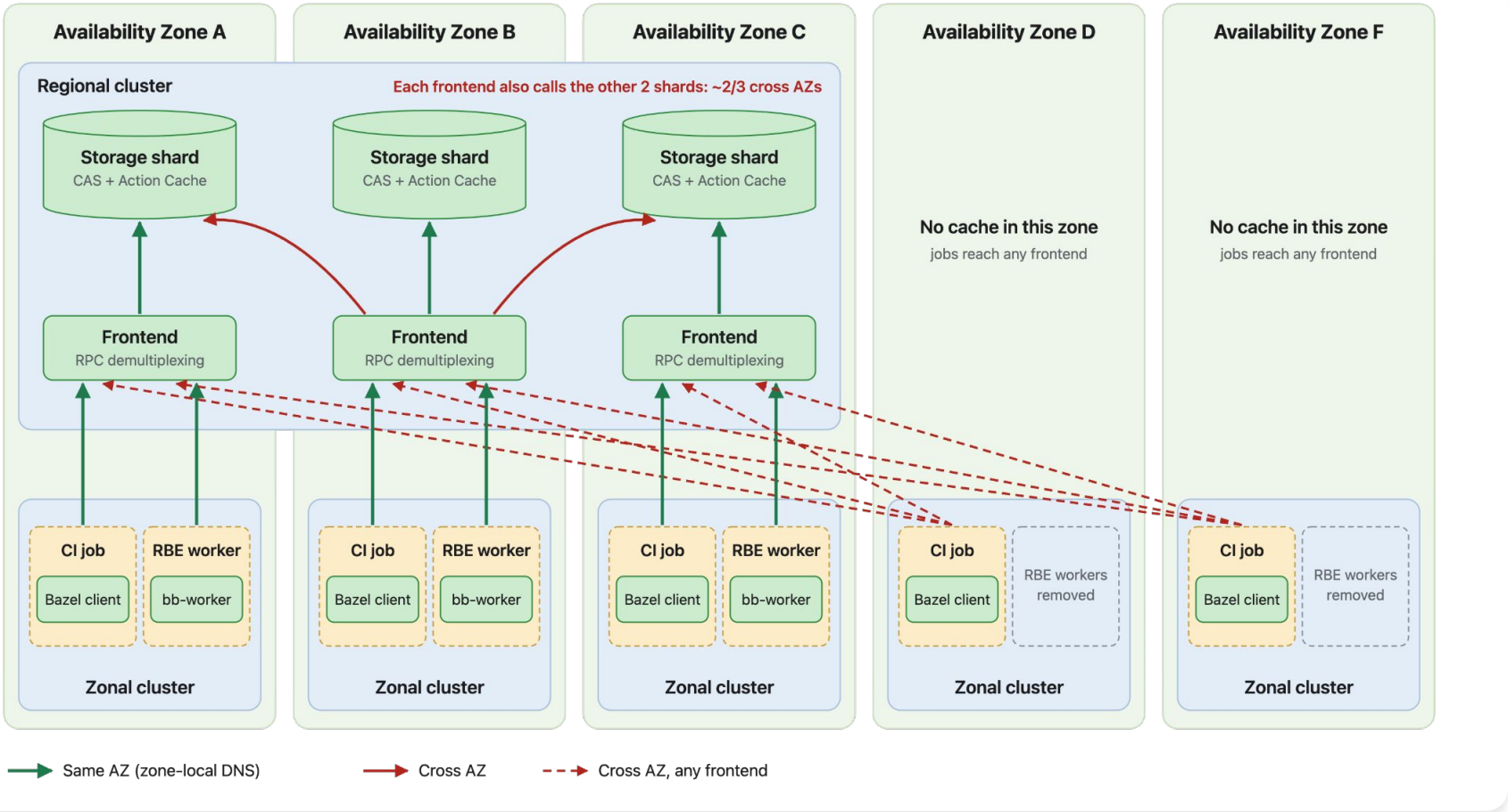


~80%
of frontend traffic
crossed zone
boundaries



Top 10
contributor to Datadog
network spend

What we did: Collocate workers with the cache instead



Scheduler Tuning

The scheduler picks the biggest hammer

Problem: Cold actions overwhelmed our large-instance pool

New actions defaulted to the largest workers.
The pool is the smallest, most expensive pool and slowest to scale.
Surges of cold actions saturated it, causing queues and job timeouts.

Solution: Default to small

Upstream fix to the `bb-remote-execution` scheduler: try the smallest unsampled worker size first, with a short timeout. Small actions finish cheaply; large actions move up.
Once every size has a sample, PageRank takes over.

PageRank: Drop the requirement to first run on the largest workers #241

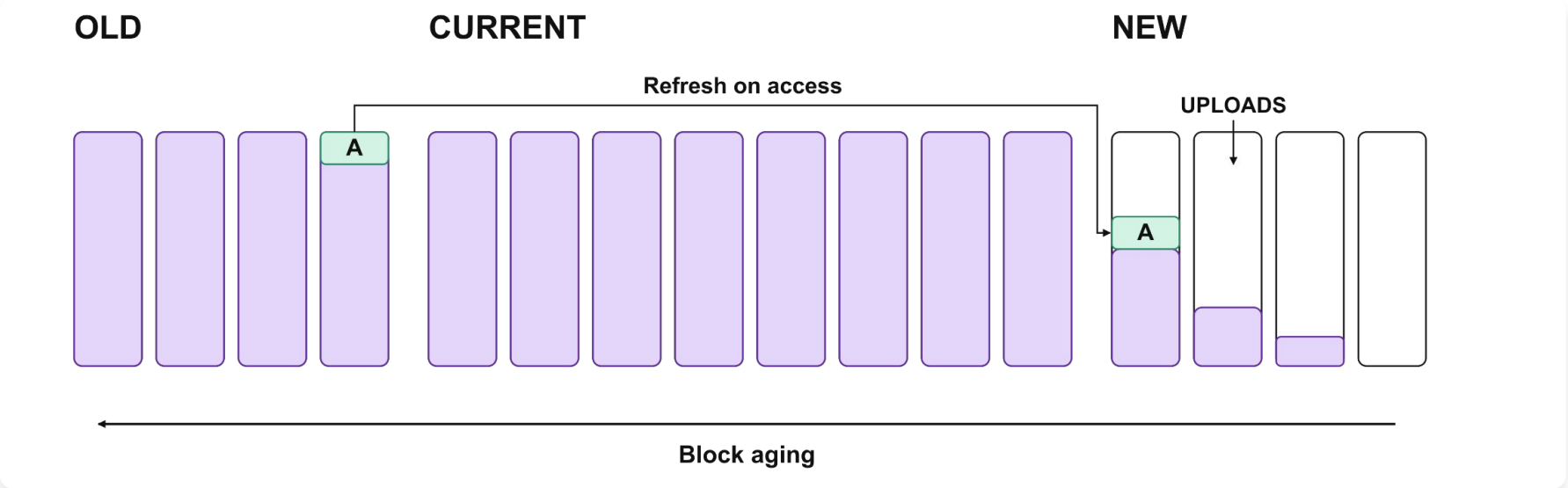
Storage contention: failure mode and mitigations

Storage contention: failure mode and mitigations

Problem: Large objects cause write pressure

1. CPU or disk contention slows request processing.
2. Connections remain open longer, with more active requests overlapping.
3. Clients continue sending data, unread data accumulates.

Block lifetime

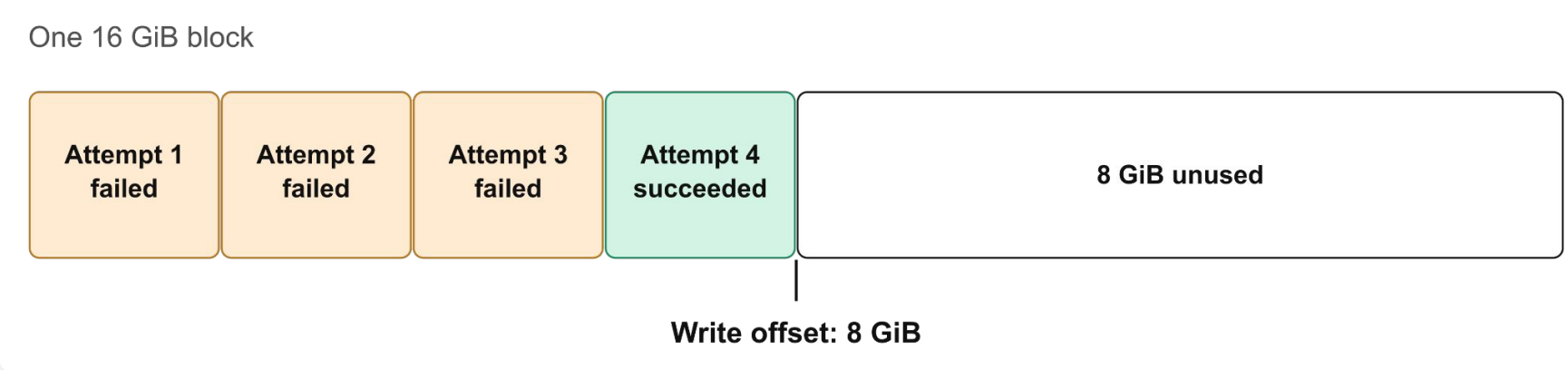


New uploads go into new blocks.

Blocks age through current and old

Accessing an old blob normally copies into new storage.

A failed write still consumes its reservation



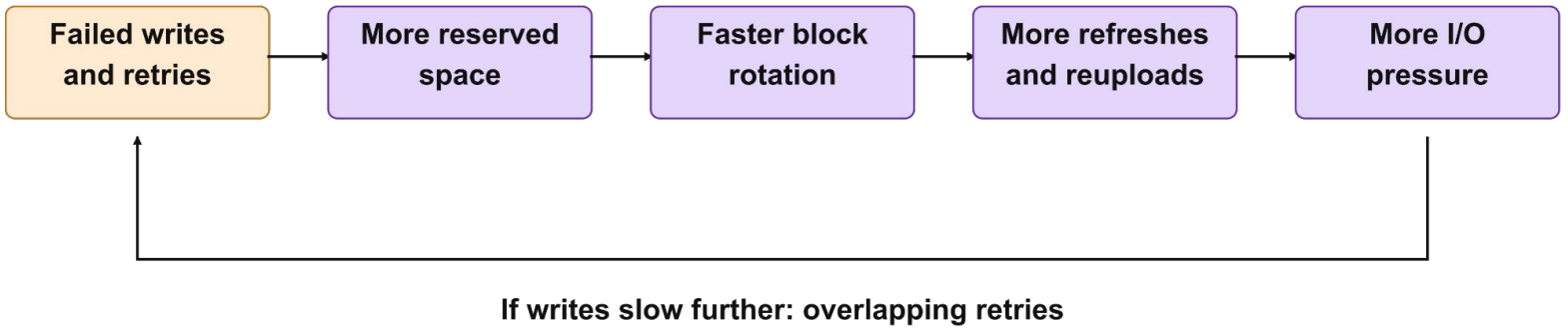
Reserve first, write second

Buildbarn reserves the blob's full size before writing it.

A failed write does not roll back that reservation.

A retry can reserve another region.

Failed writes can amplify storage pressure



A possible feedback loop

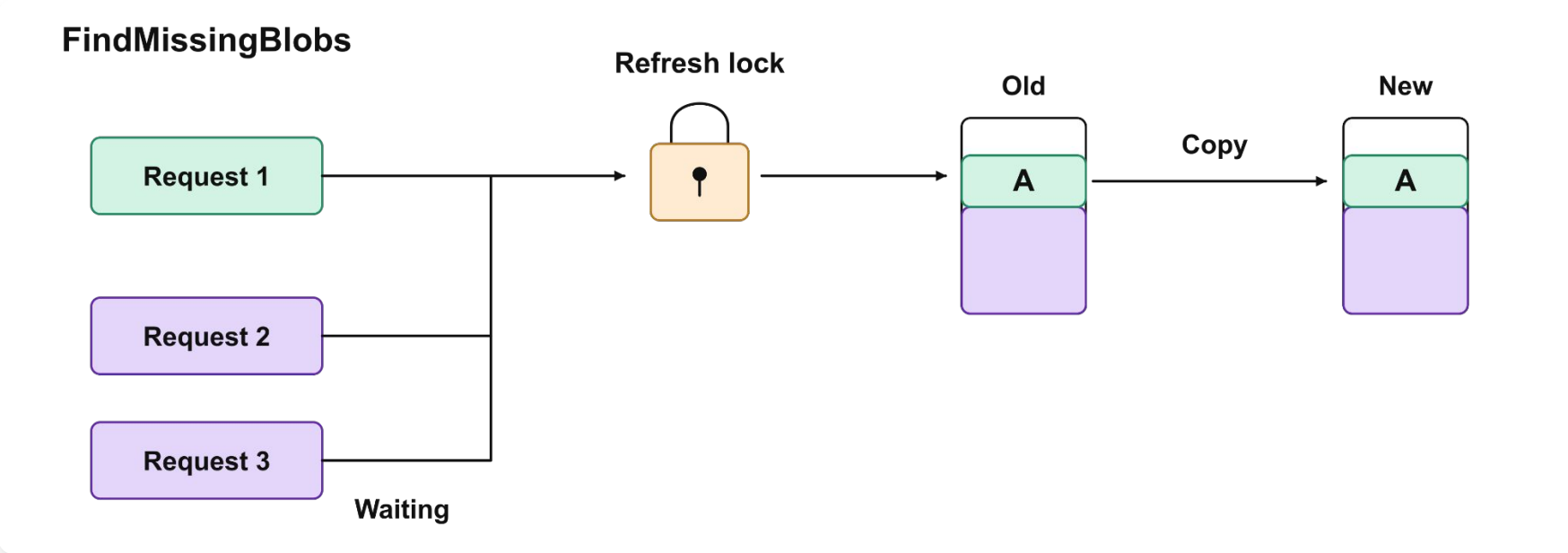
Failed writes and retries consume reservations.

Faster rotation shortens useful data retention.

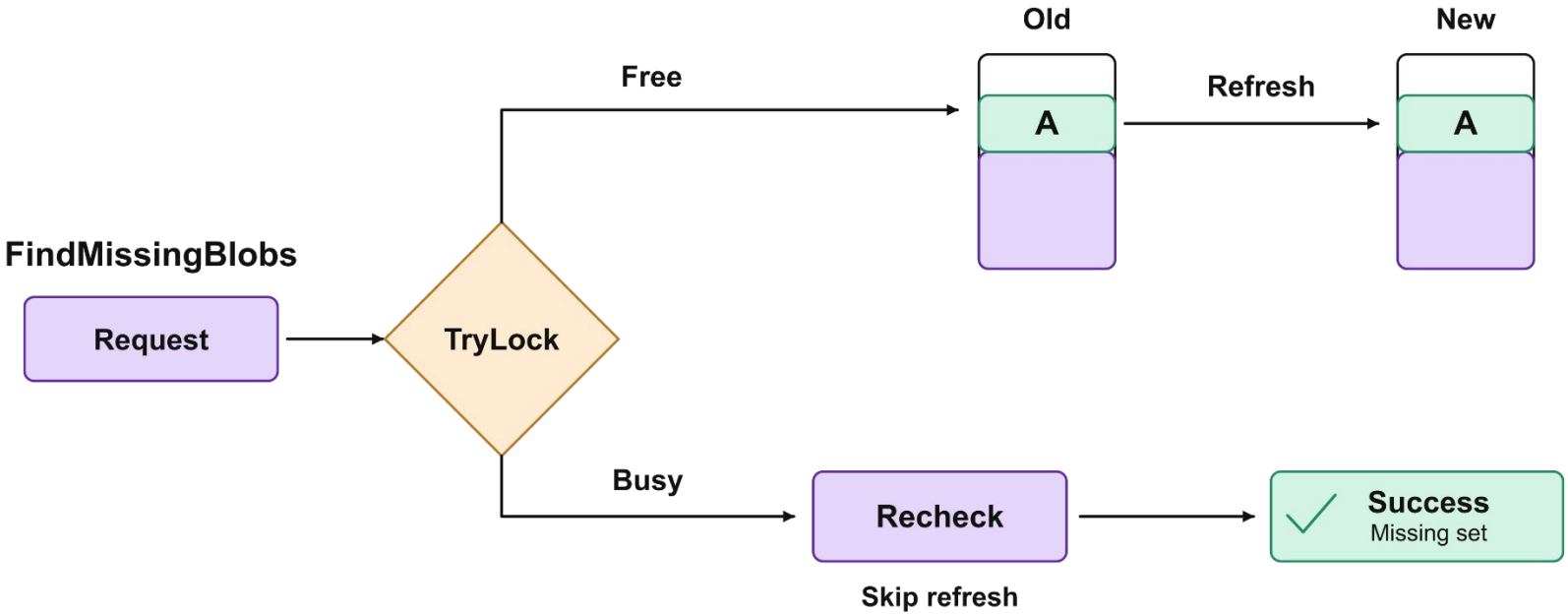
Refreshes and reuploads add more I/O.

Overlapping requests retain buffers and increase memory pressure.

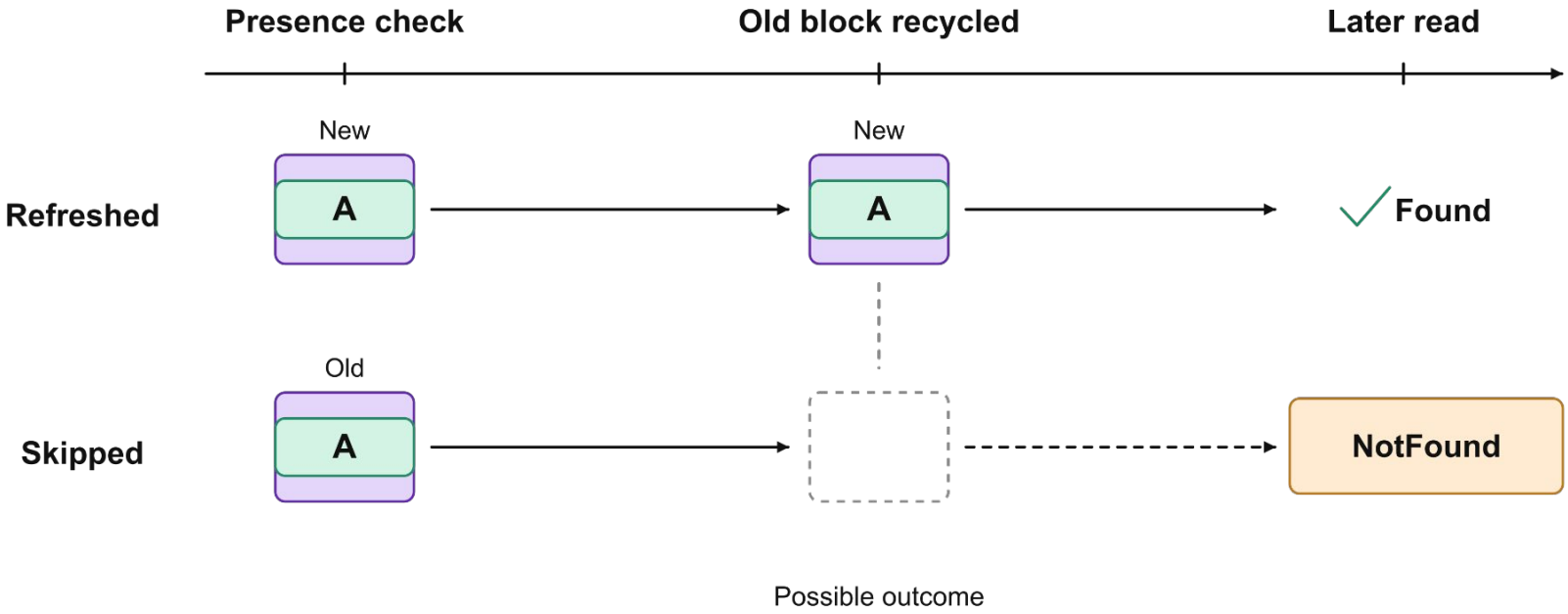
Goroutines pile up behind the FindMissing() refresh lock



Return existence without waiting for refresh



Present now can mean missing later

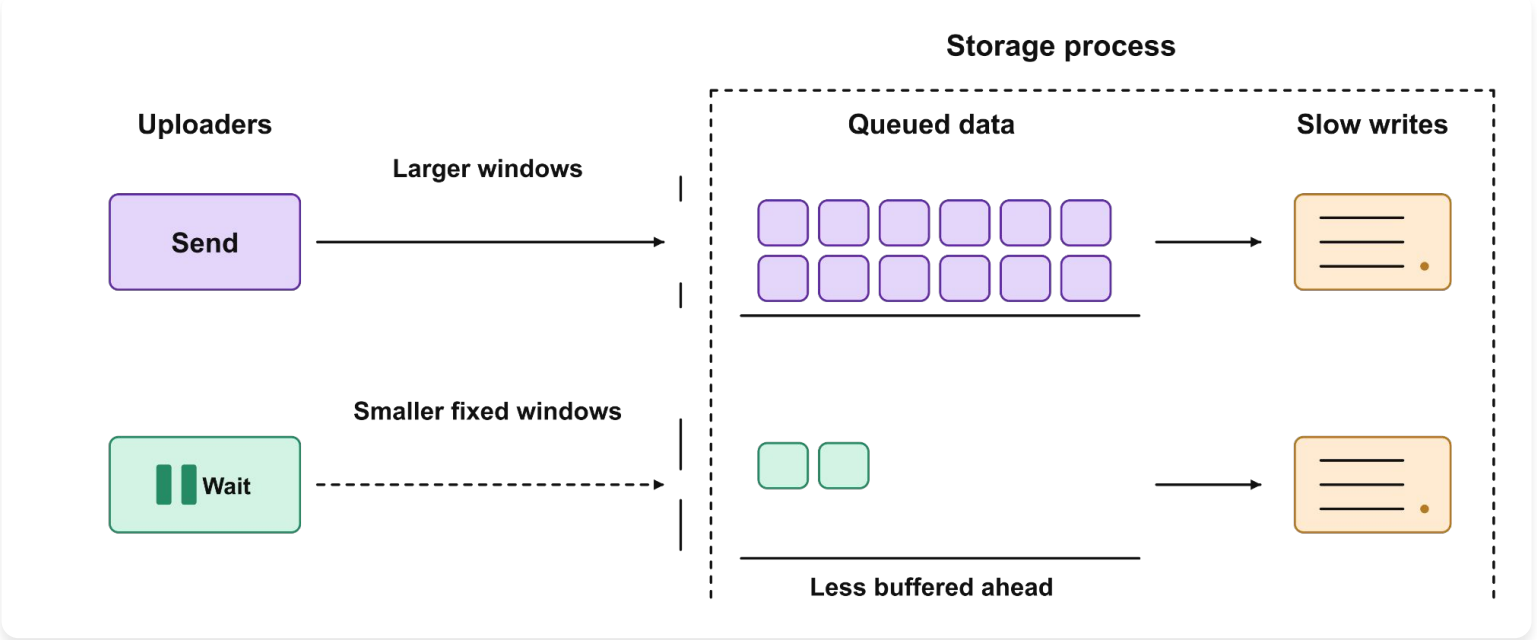


Earlier back pressure reduces upload buffering

backpressure measure for overloaded nodes: reduce how much incoming data can accumulate while storage is struggling to process it.

```
150  message ServerConfiguration {
190    // The gRPC connection's initial stream window size.  See grpc-go
191    // InitialWindowSize
192    int32 initial_window_size_bytes = 8;
193
194    // The gRPC connection's initial connection window size.  See grpc-go
195    // InitialConnWindowSize
196    int32 initial_conn_window_size_bytes = 9;
197
```

Earlier back pressure reduces upload buffering



Bazel client-side tuning

Reduce the number of actions built

Reduce the number of build pipelines

Increase the cache hit ratio

Build configuration space reduction

BEFORE

Non-reproducible builds

Cache hit rates 98.0%

Up to 3 billion Bazel actions executed in a single week

AFTER

Reproducible builds

Cache hit rates - 99.1%

Up to 2 billion Bazel actions executed in a single week

10x reduction in actions executed in RBE

Sandbox path leaks in dwarf debug info

RBE has stable paths - why do we need to fix this?

Choice of compiler -> different ways of remapping debug paths.

Rust FFI and Go CGO compilations attempt to fix this, but only partially succeed

Most likely you need a custom wrapper around your cc toolchain to fix this

Build configuration space reduction

Affects action cache hit rate

1. Turned off image builds in pre-merge CI
2. Remove use of –define for build stamping
3. Use cross compilation to target amd64 targets from arm64 build hosts over separate pipelines

What we gained



Flaky test rate

Flaky test rate dropped significantly

Dedicated resources for each test removed resource contention related flakes



Costs

CI unit costs dropped by a third



Developer UX

More consistent build times

Growth headroom from load testing

Next Steps

Thank you



DATADOG